UNIVERSIDADE FEDERAL DE SERGIPE

Método guloso

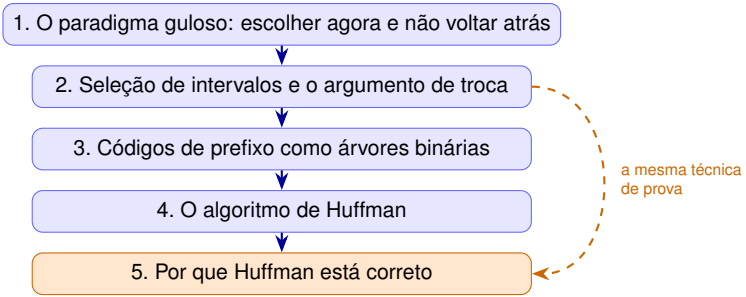
Códigos de Huffman e o argumento de troca

Prof. Dr. André Yoshiaki Kashiwabara

Departamento de Computação — Universidade Federal de Sergipe

29 de outubro de 2026

O que você vai aprender hoje



Na programação dinâmica, a decisão vinha *depois* de resolver todos os subproblemas. Hoje ela vem **antes**: uma escolha só, e um subproblema só.

Para aprofundar: Cormen et al. (2009), cap. 16 [1]; Kleinberg e Tardos (2006), cap. 4 [7].

O paradigma guloso

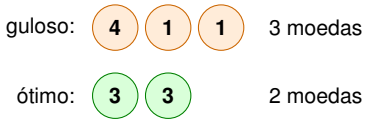
Por que não pegar sempre o melhor agora?

Problema: dar troco de c centavos com o *menor número* de moedas.

Com as moedas do real, $\{1, 5, 10, 25, 50, 100\}$, o caixa faz assim: pega a maior moeda que cabe, desconta e repete.

$c = 68$: $50 + 10 + 5 + 1 + 1 + 1$ — seis moedas, e é o ótimo.

Mas nem sempre funciona: moedas $\{1, 3, 4\}$, troco $c = 6$.



A escolha “óbvia” (a moeda 4) já descarta o ótimo, e o guloso nunca volta atrás.

A pergunta da aula: **quando** o guloso acerta, e **como provar** que acerta.

Para aprofundar: Cormen et al. (2009), Problema 16-1 (troco) [1]; Dasgupta et al. (2008), cap. 5 [4].

Método guloso: definição

Definição

Um **algoritmo guloso** constrói a solução em passos. Em cada passo, faz a escolha que parece melhor *naquele momento*, segundo um critério local, e nunca desfaz uma escolha já feita. Depois de cada escolha sobra um único subproblema, do mesmo tipo do original.

Intuição

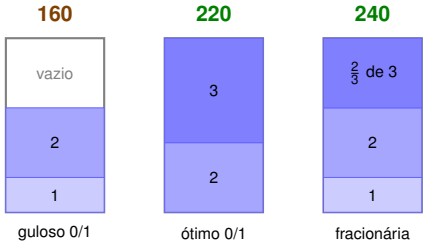
Ganhar o máximo agora, sem olhar o futuro. É rápido (em geral, ordenar e varrer uma vez), mas a escolha local só leva ao ótimo global em problemas com estrutura especial, e essa estrutura precisa ser *provada*, não suposta.

Para aprofundar: Cormen et al. (2009), introdução ao cap. 16 [1]; Kleinberg e Tardos (2006), introdução ao cap. 4 [7]; Levitin (2012), introdução ao cap. 9 [8].

Mochila: o guloso erra na 0/1 e acerta na fracionária

Capacidade $W = 50$. Itens em ordem de valor por quilo:

item	w	v	v/w
1	10	60	6
2	20	100	5
3	30	120	4



Guloso: pega o item de maior v/w que ainda cabe. Na **0/1**, os itens 1 e 2 deixam 20 kg vazios; na **fracionária**, o vazio recebe $\frac{2}{3}$ do item 3.

Por isso a mochila 0/1 pediu PD (aula de 15/10); a fracionária se resolve ordenando por v/w, em $O(n \log n)$.

Para aprofundar: Cormen et al. (2009), §16.2, Figura 16.2 [1].

Os dois ingredientes de um guloso correto

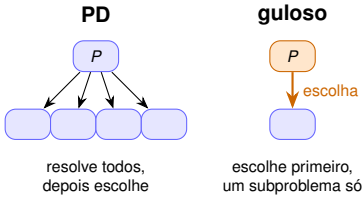
Propriedade da escolha gulosa

Existe uma solução ótima que contém a escolha gulosa. Ou seja, a escolha local pode ser feita *sem* resolver subproblemas antes.

Subestrutura ótima

Depois da escolha, o que falta da solução ótima é uma solução ótima do único subproblema que sobrou.

Para aprofundar: Cormen et al. (2009), §16.2 [1].



A subestrutura ótima é a mesma que justificou a PD. O que o guloso tem a mais é o primeiro ingrediente, e é ele que precisa de prova — quase sempre por um **argumento de troca**.

Recapitulando: o paradigma guloso

O que aprendemos

- O guloso escolhe pelo critério local e nunca volta atrás.
- Ele é rápido, mas pode errar: troco com $\{1, 3, 4\}$, mochila 0/1.
- Para estar correto, precisa da **escolha gulosa** e da **subestrutura ótima**.
- A escolha gulosa é o que a PD não precisa provar e o guloso precisa.

Para aprofundar: Cormen et al. (2009), §16.2 [1].

Seleção de intervalos e o argumento de troca

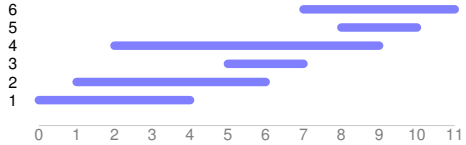
Por que voltar aos intervalos?

Problema: um auditório, n pedidos de reserva $[s_i, f_i)$. Atender o *maior número* de pedidos sem sobreposição.

Na aula de 13/10, cada pedido tinha um valor v_i e o guloso dava 7 em vez de 8: foi preciso PD.

Hoje: os mesmos seis intervalos, *sem pesos* — todo pedido vale 1.

Sem pesos, um critério guloso simples basta. Mas qual critério? Há pelo menos quatro candidatos naturais, e três deles falham.



Para aprofundar: Kleinberg e Tardos (2006), §4.1 e §6.1 [7]; Cormen et al. (2009), §16.1 [1].

Três critérios que falham

começa mais cedo



O longo começa primeiro e bloqueia os quatro curtos: 1 contra 4.

mais curto



O curto do meio corta os dois longos: 1 contra 2.

menos conflitos

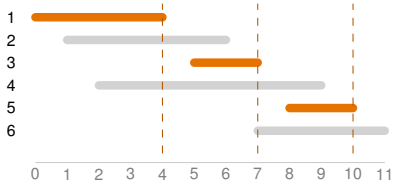


O laranja tem só 2 conflitos e é escolhido, mas elimina os dois do meio da fileira de cima: 3 contra 4.

Em laranja, a primeira escolha do critério. Um contraexemplo derruba um critério guloso; para *aceitar* um critério, é preciso uma prova.

Para aprofundar: Kleinberg e Tardos (2006), §4.1, Figura 4.1 [7].

O critério certo: termina mais cedo



Já em ordem de término $f_1 \leq f_2 \leq \cdots \leq f_n$. Varrer uma vez, escolhendo cada intervalo que começa depois do último escolhido terminar ($s_i \geq f$):

- escolhe 1, $f = 4$;
- 2 começa em 1: descarta;
- escolhe 3, $f = 7$;
- 4 começa em 2: descarta;
- escolhe 5, $f = 10$;
- 6 começa em 7: descarta.

Resultado: $\{1, 3, 5\}$, três pedidos, que é o máximo.

Ideia: terminar cedo *deixa o maior tempo livre* para os pedidos seguintes.

Para aprofundar: Cormen et al. (2009), §16.1, GREEDY-ACTIVITY-SELECTOR [1]; Kleinberg e Tardos (2006), §4.1 [7].

Seleção de intervalos em C

```c
typedef struct { int s, f; } Intervalo;

static int cmp_fim( const void *a, const void *b) {
   return ((const Intervalo *) a)->f
        - ((const Intervalo *) b)->f;
}

/* ordena v[0..n-1] por termino e grava em esc[]
   os indices escolhidos; devolve quantos sao */
int seleciona( int n, Intervalo v[], int esc[]) {
   qsort( v, n, sizeof (Intervalo), cmp_fim);
   int k = 0, fim = INT_MIN;
   for (int i = 0; i < n; ++i)
      if (v[i].s >= fim) {    /* compativel */
         esc[k++] = i;
         fim = v[i].f;
      }
   return k;
}
```

Para aprofundar: Cormen et al. (2009), §16.1 [1]; Kleinberg e Tardos (2006), §4.1 [7].

Custo. A ordenação domina: $O(n \log n)$. A varredura é $\Theta(n)$ e só guarda o término do último escolhido.

Intervalos semiabertos. $[s_i, f_i)$ e $[s_j, f_j)$ são compatíveis quando $f_i \leq s_j$: um pedido pode começar no instante em que o anterior termina.

Compare com a PD da aula de 13/10: lá eram $O(n \log n)$ para ordenar e calcular $p(j)$ e uma tabela M de $n + 1$ posições.

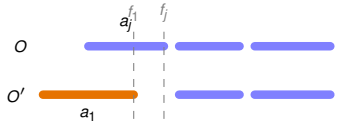
O argumento de troca

Teorema (escolha gulosa)

Seja a_1 o intervalo que termina mais cedo. Existe uma solução ótima que contém a_1.

Prova. Seja O uma solução ótima qualquer e a_j o intervalo de O que termina primeiro. Se $a_j = a_1$, acabou. Senão, troque: $O' = O - \{a_j\} \cup \{a_1\}$.

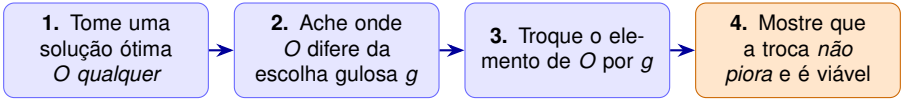
Os outros intervalos de O começam depois de f_j, e $f_1 \leq f_j$; logo nenhum deles cruza a_1. Então O' é compatível e $|O'| = |O|$: O' também é ótima e contém a_1. □



Com a subestrutura ótima (o resto de O' é ótimo para os intervalos que começam depois de f_1), uma indução no número de intervalos prova que o guloso é ótimo.

Para aprofundar: Cormen et al. (2009), §16.1, Teorema 16.1 [1]; Kleinberg e Tardos (2006), §4.1 [7].

O roteiro de um argumento de troca



Conclusão: existe solução ótima que concorda com o guloso na primeira escolha. Repetindo (indução), existe uma que concorda em *todas*: a solução gulosa é ótima.

Onde vamos usar: hoje, na corretude de Huffman (trocar folhas de posição na árvore); nas próximas aulas, em Kruskal e Prim (trocar arestas de uma árvore geradora) e em Dijkstra.

Para aprofundar: Kleinberg e Tardos (2006), §4.2 (argumento de troca) [7]; Cormen et al. (2009), §16.2 [1].

Recapitulando: seleção de intervalos

O que aprendemos

- Sem pesos, “termina mais cedo” resolve a seleção de intervalos em $O(n \log n)$.
- “Começa mais cedo”, “mais curto” e “menos conflitos” caem com contraexemplos.
- O **argumento de troca** transforma uma solução ótima qualquer em uma que contém a escolha gulosa, sem piorá-la.
- Com pesos, a troca deixa de funcionar e voltamos à PD.

Para aprofundar: Cormen et al. (2009), §16.1 [1]; Kleinberg e Tardos (2006), §4.1–4.2 [7].

Códigos de prefixo

Por que códigos de tamanho variável?

Problema: um arquivo de 100 mil caracteres com só seis letras. Quantos bits são necessários para guardá-lo?

Com um código de tamanho fixo, cada letra gasta $\lceil \log_2 6 \rceil = 3$ bits: 300 mil bits.

Ideia: letras frequentes ganham palavras curtas, como no código Morse.

	a	b	c	d	e	f
freq. (mil)	45	13	12	16	9	5
fixo	000	001	010	011	100	101
variável	0	101	100	111	1101	1100

Com o código variável: $45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4 = 224$ mil bits, uma economia de 25%. Esse é o exemplo condutor do resto da aula.

Para aprofundar: Cormen et al. (2009), §16.3, Figura 16.3 [1].

Código de prefixo: definição

Definição

Um **código de prefixo** atribui a cada caractere uma palavra binária de modo que nenhuma palavra é prefixo de outra.

Intuição

Sem prefixos em comum, a decodificação nunca hesita: lendo os bits da esquerda para a direita, a primeira palavra completa encontrada só pode ser a certa.

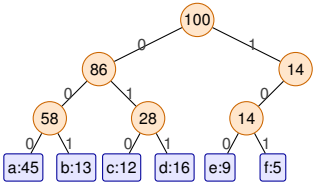
Contraexemplo: com $\mathrm{a} = 0$, $\mathrm{b} = 01$ e $\mathrm{c} = 1$, a sequência 01 pode ser “b” ou “ac”. O código variável do slide anterior é de prefixo: 0101100 só pode ser $0\,101\,100 =$ “abc”.

Pode-se provar que nenhum código decodificável (de prefixo ou não) comprime melhor que o melhor código de prefixo, então basta procurar entre eles.

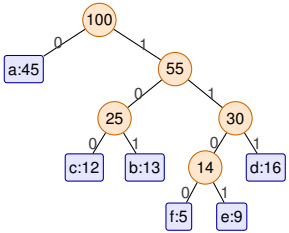
Para aprofundar: Cormen et al. (2009), §16.3 [1]; Cover e Thomas (2006), §5.1 e §5.5 [3].

Códigos como árvores binárias

código fixo



código variável



Cada caractere é uma folha; a palavra é o caminho da raiz até ela (0 = esquerda, 1 = direita). Ser de prefixo é exatamente isto: *nenhum caractere em nó interno*. Cada nó interno mostra a soma das frequências abaixo dele. A árvore fixa tem um nó interno com um filho só: dá para encurtar e e f.

Para aprofundar: Cormen et al. (2009), §16.3, Figura 16.4 [1].

O custo de uma árvore

Definição

Para um alfabeto C com frequências $c.freq$ e uma árvore de código T, em que $d_T(c)$ é a profundidade da folha c (o tamanho da sua palavra),

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c) \quad \text{bits.}$$

O problema: dada C, achar T que **minimiza** $B(T)$ — um código ótimo.

Fato 1. Uma árvore ótima é *cheia*: todo nó interno tem dois filhos. (Um nó com filho único pode ser contraído, e as palavras abaixo dele encurtam.)

Fato 2. $B(T)$ é a soma dos valores dos nós internos, pois a frequência de uma folha é somada uma vez em cada ancestral. Na árvore variável:
$14 + 25 + 30 + 55 + 100 = 224$.

Para aprofundar: Cormen et al. (2009), §16.3, Equação (16.4) [1]; Levitin (2012), §9.4 [8].

Recapitulando: códigos de prefixo

O que aprendemos

- Palavras curtas para caracteres frequentes reduzem o total de bits.
- Um código de prefixo decodifica sem ambiguidade e corresponde a uma árvore binária com os caracteres nas folhas.
- O custo é $B(T) = \sum_c c.freq \cdot d_T(c)$; a árvore ótima é cheia.
- No exemplo: 300 mil bits com código fixo, 224 mil com o variável.

Para aprofundar: Cormen et al. (2009), §16.3 [1].

O algoritmo de Huffman

Por que construir de baixo para cima?

Problema: há uma quantidade exponencial de árvores cheias com n folhas. Testar todas está fora de questão.

Uma observação ajuda: os dois caracteres *menos* frequentes deveriam ficar no fundo da árvore, onde as palavras são mais longas.

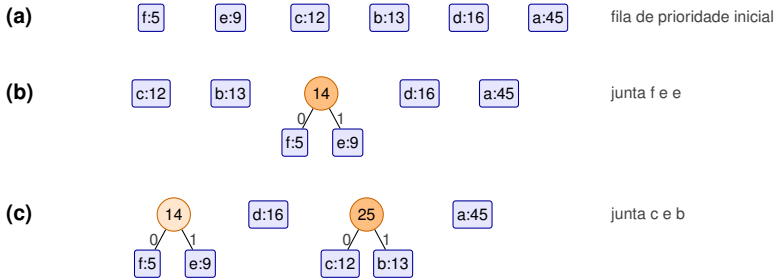
Solução (Huffman, 1952): começar com n árvores de um nó só e repetir $n - 1$ vezes:

1. tire as duas árvores de **menor** frequência, x e y;
2. crie um nó z com filhos x e y e $z.freq = x.freq + y.freq$;
3. devolva z ao conjunto.

A escolha gulosa: juntar sempre os dois menores.

Para aprofundar: Huffman (1952) [6]; Cormen et al. (2009), §16.3, HUFFMAN [1]; Dasgupta et al. (2008), §5.2 [4].

Exemplo: as primeiras fusões

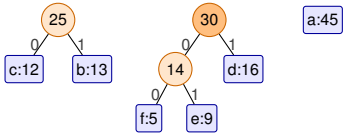


Em cada linha, a fila em ordem crescente de frequência; em laranja, o nó recém-criado. Na linha (b), o nó 14 entra *entre* b:13 e d:16: a fila é reordenada a cada fusão, por isso usamos um heap.

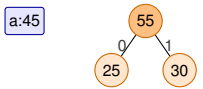
Para aprofundar: Cormen et al. (2009), §16.3, Figura 16.5(a)–(c) [1].

Exemplo: as últimas fusões

(d) junta 14 e d

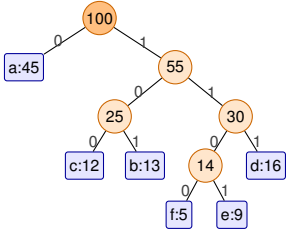


(e) junta 25 e 30



(subárvores como em (d))

(f) junta a e 55: a árvore final



$n = 6$ caracteres, $n - 1 = 5$ fusões. Os nós internos valem $14, 25, 30, 55, 100$, e a soma é $B(T) = 224$ mil bits: a árvore variável do início da aula era a de Huffman. Palavras: a = 0, c = 100, b = 101, f = 1100, e = 1101, d = 111.

Para aprofundar: Cormen et al. (2009), §16.3, Figura 16.5(d)–(f) [1].

Huffman em C

```
/* nos 0..n-1 sao as folhas (freq[] dado);
   nos n..2n-2 sao os internos criados aqui.
   Heap: fila de prioridade minima de AED 1. */
int huffman( int n, int freq[], int esq[], int dir[]) {
   Heap h = heap_cria( n);
   for (int i = 0; i < n; ++i)
      heap_insere( &h, i, freq[i]);
   for (int z = n; z < 2*n - 1; ++z) { /* n-1 fusoes */
      int x = heap_remove_min( &h);
      int y = heap_remove_min( &h);
      esq[z] = x;  dir[z] = y;
      freq[z] = freq[x] + freq[y];
      heap_insere( &h, z, freq[z]);
   }
   return heap_remove_min( &h);  /* a raiz, 2n-2 */
}
```

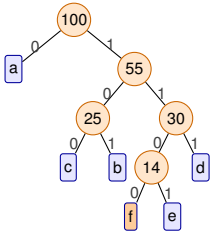
Custo. São $n - 1$ iterações, cada uma com três operações no heap de até n elementos: $O(n \log n)$ no total. (Construir o heap inicial com *build-heap* custa $O(n)$.)

Árvore em vetores. Os nós são índices; `esq[z]` e `dir[z]` são os filhos do nó interno `z`. As palavras saem de uma DFS da raiz que acrescenta 0 ao descer à esquerda e 1 à direita.

Com as frequências já ordenadas, duas filas simples substituem o heap: $O(n)$.

Para aprofundar: Cormen et al. (2009), §16.3, HUFFMAN [1]; van Leeuwen (1976), duas filas [10]; Levitin (2012), §9.4 [8].

Decodificando com a árvore



Começar na raiz; para cada bit, descer à esquerda (0) ou à direita (1). Ao chegar numa folha, emitir o caractere e voltar à raiz.

bits lidos	caminho	emite
0	esquerda	a
101	dir., esq., dir.	b
1100	dir., dir., esq., esq.	f
1101	dir., dir., esq., dir.	e

Assim 010111001101 = “abfe”. Cada bit custa $O(1)$: decodificar é linear no tamanho do texto codificado.

Para aprofundar: Cormen et al. (2009), §16.3 [1]; Kleinberg e Tardos (2006), §4.8 [7].

Recapitulando: o algoritmo de Huffman

O que aprendemos

- Huffman junta, $n - 1$ vezes, as duas árvores de menor frequência.
- Com um heap, custa $O(n \log n)$.
- No exemplo, produz $B(T) = 224$ mil bits, a soma dos nós internos.
- Decodificar é descer na árvore, um bit por passo.

Falta a pergunta central: por que juntar os dois menores dá a árvore *ótima*?

Para aprofundar: Cormen et al. (2009), §16.3 [1].

Por que Huffman está correto

Por que precisamos de uma prova?

Problema: Huffman é guloso. Vimos três critérios gulosos “razoáveis” para intervalos falharem, e o guloso da mochila 0/1 também falhou.

Um bom exemplo não é prova: precisamos dos dois ingredientes.

Solução: dois lemas, um para cada ingrediente.

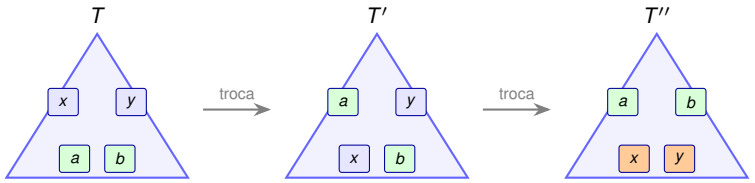
- **Escolha gulosa** (Lema 16.2): juntar os dois menores não exclui o ótimo — argumento de troca.
- **Subestrutura ótima** (Lema 16.3): depois da fusão, o resto é um problema de Huffman menor.

Para aprofundar: Cormen et al. (2009), §16.3, Lemas 16.2 e 16.3 [1].

Lema da escolha gulosa

Lema

Sejam x e y os dois caracteres de menor frequência de C. Existe um código de prefixo ótimo em que x e y são **irmãos** na árvore: suas palavras têm o mesmo tamanho e diferem só no último bit.



T é uma árvore ótima qualquer; a e b são folhas irmãs de profundidade máxima (existem porque T é cheia). Trocar x com a e y com b leva x, y ao fundo, como irmãos.

Para aprofundar: Cormen et al. (2009), §16.3, Lema 16.2 e Figura 16.6 [1].

Prova: a troca não piora

Só as parcelas de x e a mudam de T para T':

$$\begin{aligned} B(T) - B(T') &= x.freq\, d_T(x) + a.freq\, d_T(a) - x.freq\, d_T(a) - a.freq\, d_T(x) \\ &= \underbrace{(a.freq - x.freq)}_{\geq 0:\ x \text{ é mínimo}} \underbrace{(d_T(a) - d_T(x))}_{\geq 0:\ a \text{ é o mais fundo}} \geq 0. \end{aligned}$$

Do mesmo modo, $B(T') - B(T'') \geq 0$. Logo $B(T'') \leq B(T)$; como T é ótima, $B(T'') = B(T)$ e T'' também é ótima, com x e y irmãos. □

É o roteiro da seção 2: uma solução ótima qualquer, uma troca que aproxima da escolha gulosa, e a conta mostrando que não piora. A frequência baixa vai para o fundo, a alta sobe.

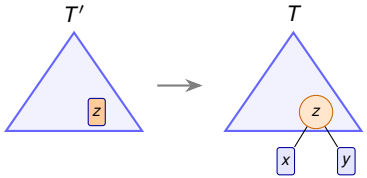
Para aprofundar: Cormen et al. (2009), §16.3, prova do Lema 16.2 [1]; Kleinberg e Tardos (2006), §4.8 [7].

Subestrutura ótima e o teorema

Lema

Seja $C' = C - \{x, y\} \cup \{z\}$, com $z.freq = x.freq + y.freq$. Se T' é ótima para C', então a árvore T obtida trocando a folha z por um nó com filhos x e y é ótima para C.

Chave da prova: $B(T) = B(T') + x.freq + y.freq$, porque x e y estão um nível abaixo de onde z estava. Se houvesse T^* melhor para C, por (16.2) podemos supor x, y irmãos em T^*; contraí-los daria uma árvore para C' melhor que T', absurdo.



Teorema

Huffman produz um código de prefixo ótimo.

Indução em $n = |C|$: a primeira fusão é segura (16.2) e o que sobra é Huffman em C', com $n - 1$ caracteres (16.3).

Para aprofundar: Cormen et al. (2009), §16.3, Lema 16.3 e Teorema 16.4 [1].

Recapitulando: corretude de Huffman

O que aprendemos

- **Escolha gulosa:** trocar folhas leva os dois menos frequentes ao fundo sem aumentar $B(T)$.
- **Subestrutura ótima:** fundir x e y em z custa exatamente $x.freq + y.freq$ a mais.
- Indução sobre o número de caracteres fecha a prova.
- É o mesmo esqueleto da seleção de intervalos.

Para aprofundar: Cormen et al. (2009), §16.3 [1].

Fechando

Quão perto do limite Huffman chega?

Com p_c a fração de ocorrências de c, a **entropia**

$$H = -\sum_c p_c \log_2 p_c$$

é o mínimo de bits por caractere que *qualquer* código decodificável consegue em média, para caracteres independentes. O comprimento médio L do código de Huffman satisfaz

$$H \leq L < H + 1.$$

No exemplo:

	bits/caractere
código fixo	3,00
Huffman (L)	2,24
entropia (H)	2,22

Huffman é usado no estágio final de formatos como DEFLATE (`zip`, `gzip`, PNG) e JPEG, depois de etapas que exploram repetições e correlações.

Para aprofundar: Shannon (1948) [9]; Cover e Thomas (2006), §5.4, Teorema 5.4.1, e §5.8 [3]; Deutsch (1996), RFC 1951 [5].

Guloso ou programação dinâmica?

problema	técnica	por quê	tempo
intervalos sem peso	guloso	troca: termina mais cedo	$O(n \log n)$
intervalos ponderados	PD	a troca falha com pesos	$O(n \log n)$
mochila fracionária	guloso	troca: maior v/w	$O(n \log n)$
mochila 0/1	PD	guloso perde (160 contra 220)	$\Theta(nW)$
códigos de prefixo	guloso	troca de folhas (Huffman)	$O(n \log n)$
LCS, edição, Floyd	PD	sem escolha local segura	$\Theta(mn)$, $\Theta(n^3)$

Os dois métodos exigem subestrutura ótima. O guloso exige, além disso, uma escolha que possa ser feita antes de resolver os subproblemas — e, quando ela existe, troca uma tabela inteira por uma única varredura.

Próximas aulas: árvore geradora mínima (Kruskal e Prim) e caminhos mínimos (Dijkstra), dois gulosos em grafos provados por troca.

Para aprofundar: Cormen et al. (2009), §16.2 [1]; Kleinberg e Tardos (2006), cap. 4 e 6 [7].

Referências I



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Introduction to Algorithms.
MIT Press, 3 edition, 2009.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
Algoritmos: Teoria e Prática.
Elsevier, Rio de Janeiro, 3 edition, 2012.
Tradução da 3. ed. norte-americana. Seção 4.5.



Thomas M. Cover and Joy A. Thomas.
Elements of Information Theory.
Wiley, 2 edition, 2006.
Cap. 5: §5.4 (limites do comprimento ótimo, Teorema 5.4.1), §5.5 (desigualdade de Kraft–McMillan), §5.8 (otimalidade de Huffman).



Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani.
Algorithms.
McGraw-Hill, 2008.
Seção 2.1: Multiplication (o truque de Gauss).



L. Peter Deutsch.
DEFLATE compressed data format specification version 1.3.
RFC 1951, Internet Engineering Task Force, 1996.

Referências II



David A. Huffman.

A method for the construction of minimum-redundancy codes.
Proceedings of the IRE, 40(9):1098–1101, 1952.



Jon Kleinberg and Éva Tardos.

Algorithm Design.
Pearson/Addison-Wesley, 2006.
Capítulo 5, em especial as seções 5.2 (recorrências) e 5.4 (par mais próximo).



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.
Pearson, 3 edition, 2012.



Claude E. Shannon.

A mathematical theory of communication.
Bell System Technical Journal, 27(3):379–423, 1948.



Jan van Leeuwen.

On the construction of Huffman trees.
In *Proceedings of the 3rd International Colloquium on Automata, Languages and Programming (ICALP)*, pages 382–410. Edinburgh University Press, 1976.